Dossier de conception détaillée

API REST de gestion d’épargne à objectif

LICELI Technologies

Stage technicien — DIPAMA Gamine — 2026

Sommaire

Sommaire 2

1. Objet du document 3

2. Diagrammes de séquence 3

3. Diagramme d'état-transition (cycle de vie d'une tirelire) 9

4. Détail des classes et de leurs méthodes 10

5. Gestion des exceptions métier 12

6. Contrat d'interface (API) 12

7. Matrice de traçabilité (vers les tests unitaires) 15

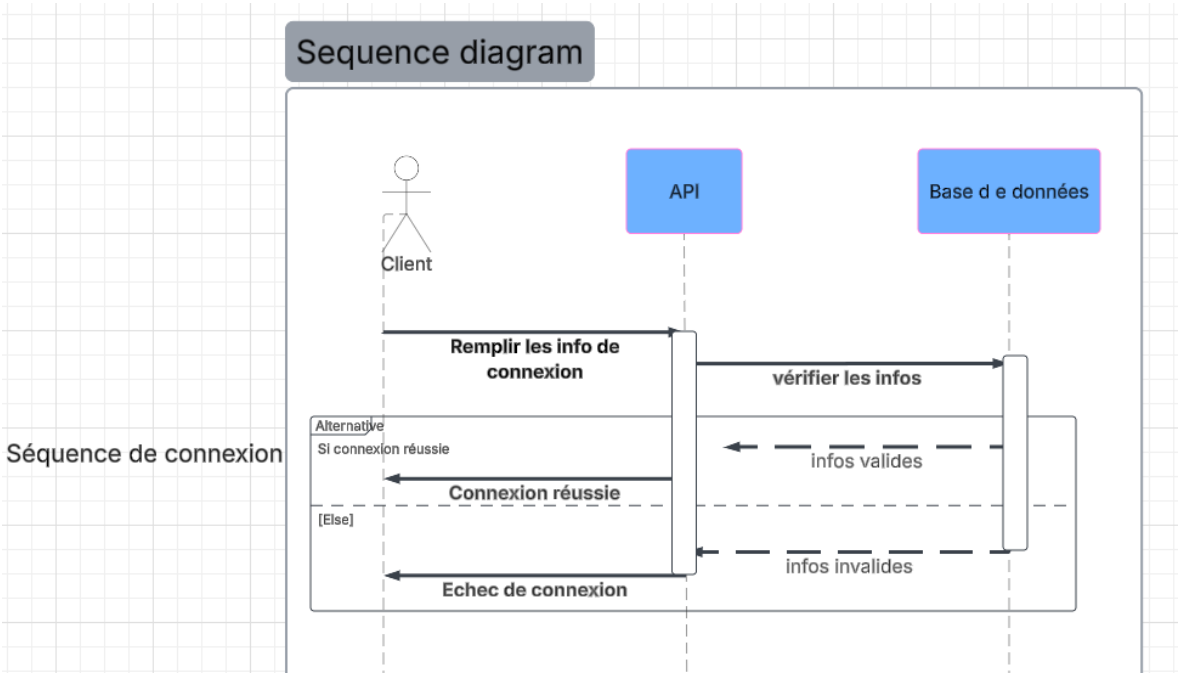
Table des matières 16

1. Objet du document

Ce document détaille, au niveau le plus fin, le comportement précis de chaque fonctionnalité : enchaînement exact des messages entre les composants (diagrammes de séquence), cycle de vie des objets métier (diagramme d'état-transition), signature et responsabilité de chaque méthode, gestion des exceptions, et contrat d'interface complet de l'API. Il sert de référence directe à l'écriture du code et aux tests unitaires.

2. Diagrammes de séquence

2.1 Séquence de connexion



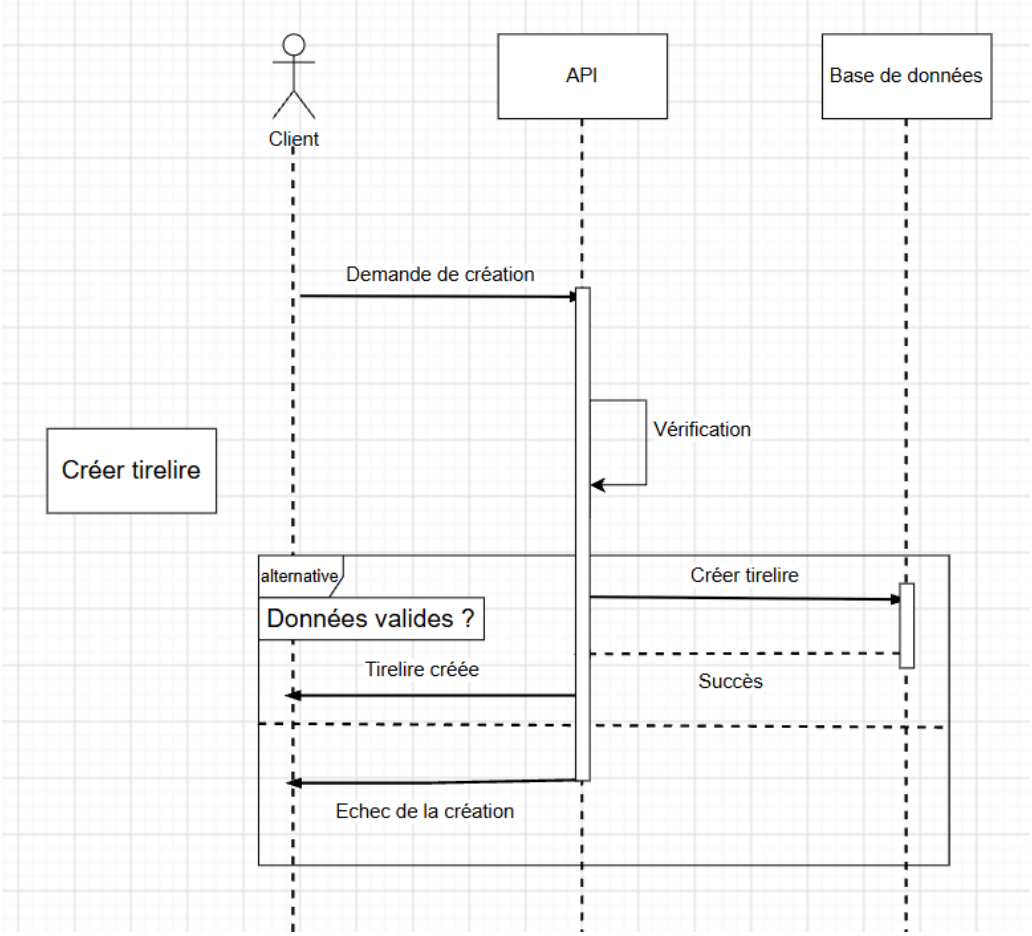
Acteurs impliqués : Client, API, Base de données

Déroulement :

- Le client transmet ses informations de connexion à l'API.
- L'API transmet ces informations à la base de données pour vérification.
- La base de données retourne le résultat de la vérification (informations valides ou invalides).
- Si les informations sont valides **:** l'API retourne au client une confirmation de connexion.
- Sinon **:** l'API retourne un message d'échec de connexion, sans préciser si c'est le nom d'utilisateur ou le mot de passe qui est erroné.

Règle de gestion associée : Aucune des RG1 à RG4.

2.2 Séquence de création d'une tirelire



Acteurs impliqués : Client, API, Base de données

Préconditions : Le client est authentifié. Chaque requête porte un jeton d'accès valide, vérifié par l'API avant tout traitement.

Déroulement :

- Le client transmet une demande de création (libellé, montant objectif, date d'échéance).
- L'API effectue une vérification interne des données transmises (montant objectif strictement positif, date d'échéance dans le futur).
- Si les données sont valides :

  a. L'API demande à la base de données la création de la tirelire.

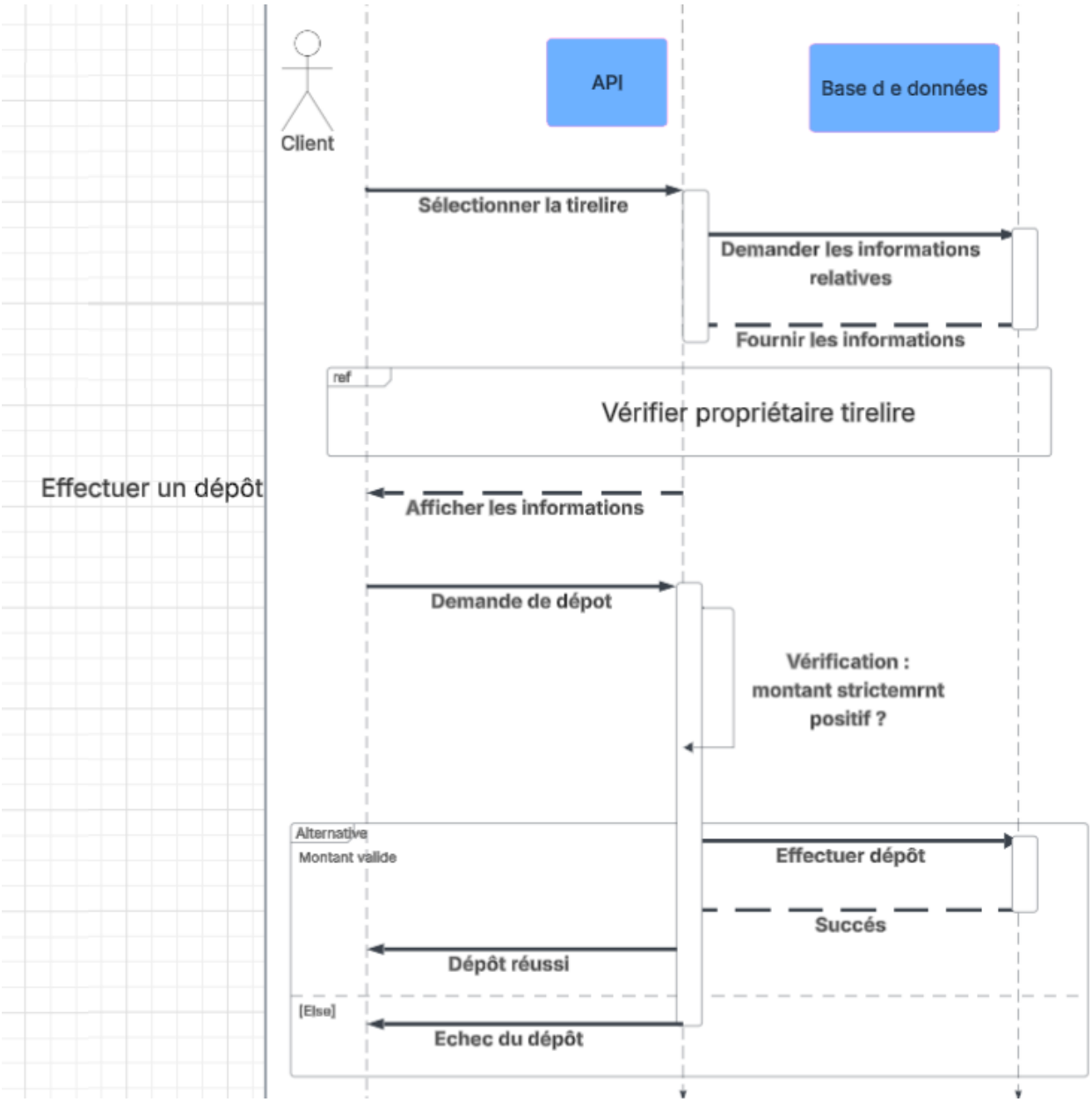
b. La base de données confirme la création.

c. L'API retourne au client la confirmation, avec les informations de la tirelire créée.

- Sinon : l'API retourne directement un message d'échec de la création, explicitant la donnée invalide, sans solliciter la base de données.

Point de conception important : la vérification des données conditionne strictement l'appel à la base de données. Aucune tentative de création n'est effectuée tant que les données n'ont pas été validées par l'API.

2.3 Séquence d'effectuer un dépôt



Acteurs impliqués : Client, API, Base de données

Préconditions : Le client est authentifié. Chaque requête porte un jeton d’accès valide, vérifié par l’API avant tout traitement.

Déroulement :

- Le client sélectionne une tirelire ; l'API récupère et affiche les informations de celle-ci auprès de la base de données.
- Référence **:** l'API s'assure que la tirelire appartient bien au client connecté (RG4).
- Le client transmet une demande de dépôt, avec le montant souhaité.
- L'API vérifie que le montant transmis est strictement positif (RG1).
- Si le montant est valide :

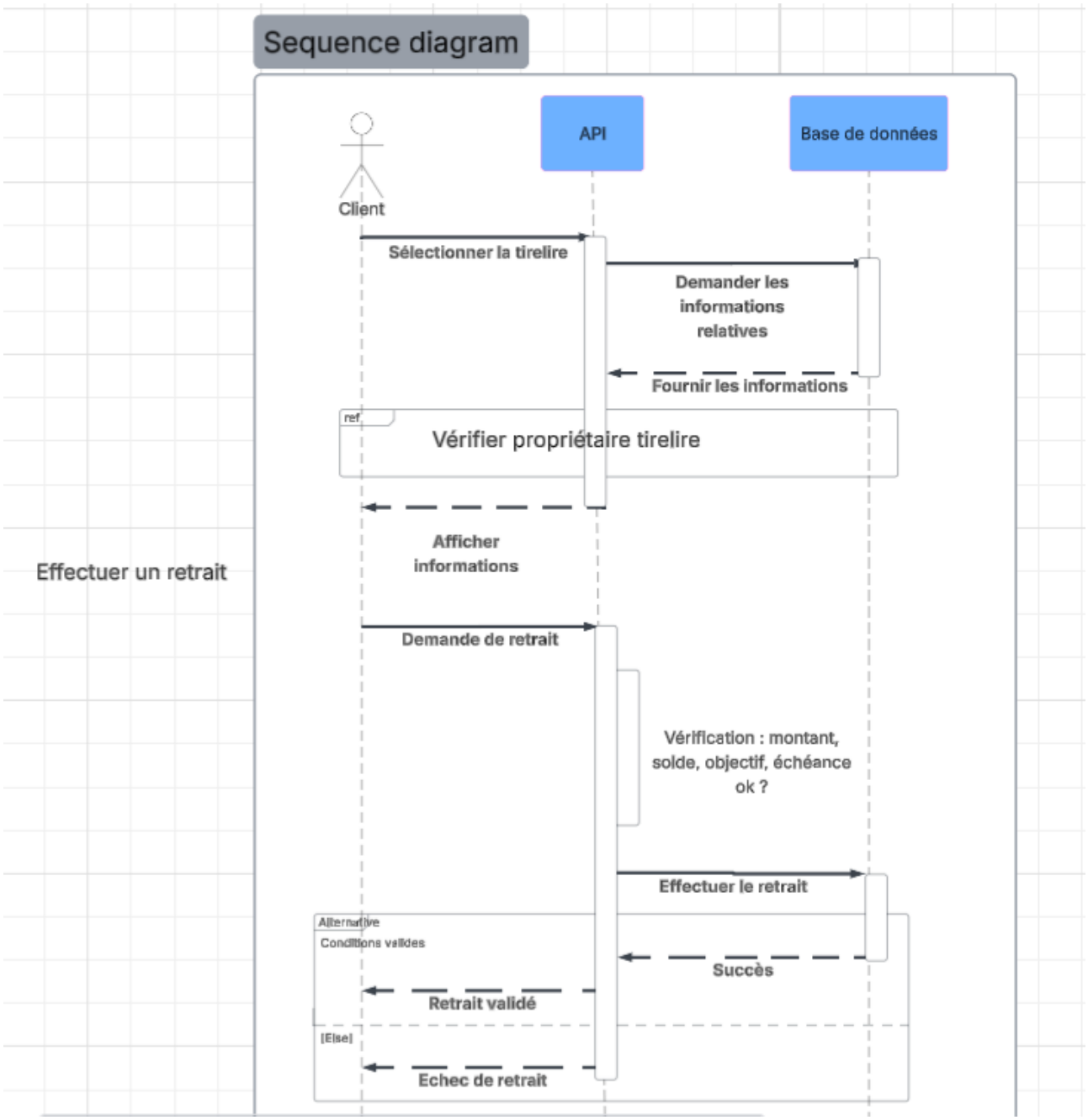
  a. L'API demande à la base de données d'enregistrer l'opération de dépôt et de mettre à jour le solde.

  b. La base de données confirme l'opération.

  c. L'API retourne au client la confirmation du dépôt et le nouveau solde.
- Sinon **:** l'API retourne un message d'échec, précisant que le montant est invalide.

Règles de gestion associées : RG1, RG4.

2.4 Séquence d’effectuer un retrait



Acteurs impliqués : Client, API, Base de données

Préconditions : Le client est authentifié. Chaque requête porte un jeton d’accès valide, vérifié par l’API avant tout traitement.

Déroulement :

- Le client sélectionne une tirelire ; l'API récupère et affiche les informations de celle-ci auprès de la base de données.
- Le client transmet une demande de retrait, avec le montant souhaité.
- Référence **:** l'API s'assure que la tirelire appartient bien au client connecté (RG4).
- L'API vérifie, dans l'ordre : que le montant transmis est strictement positif (RG1), que le solde disponible est suffisant pour couvrir le retrait (RG2), et que l'objectif est atteint ou l'échéance dépassée (RG3).
- Si les trois conditions sont réunies :

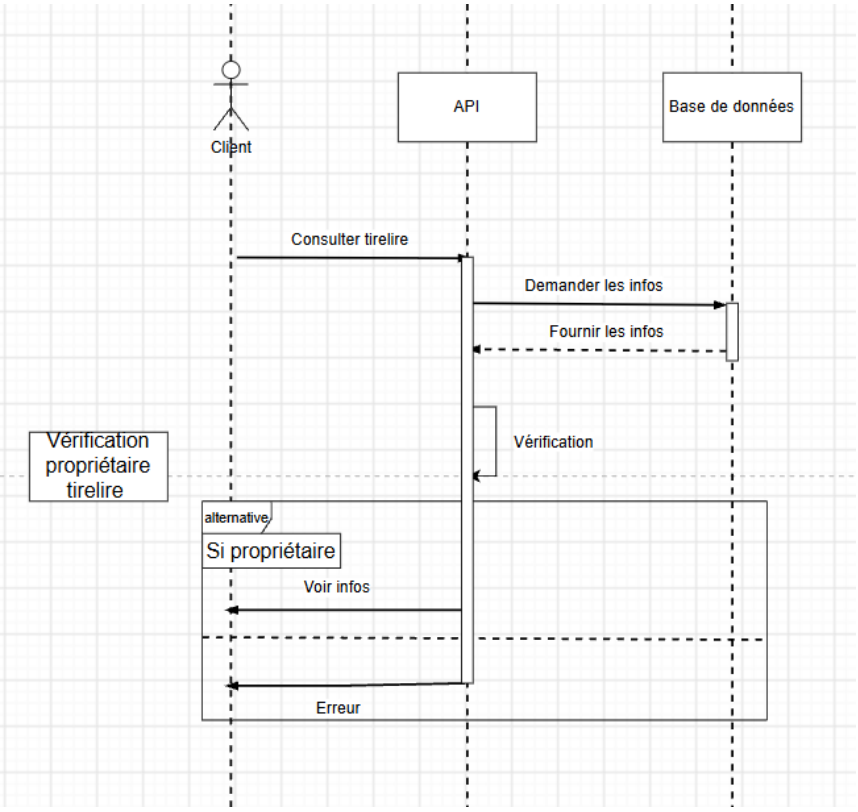
  a. L'API demande à la base de données d'enregistrer l'opération de retrait et de mettre à jour le solde.

  b. La base de données confirme l'opération.

  c. L'API retourne au client la confirmation du retrait et le nouveau solde.
- Sinon **:** l'API retourne un message d'échec, précisant laquelle des trois règles n'est pas respectée.

Règles de gestion associées : RG1, RG2, RG3, RG4.

2.5 Séquence de vérification du propriétaire d'une tirelire



Acteurs impliqués : Client, API, Base de données

Préconditions : Le client est authentifié. Chaque requête porte un jeton d'accès valide, vérifié par l'API avant tout traitement.

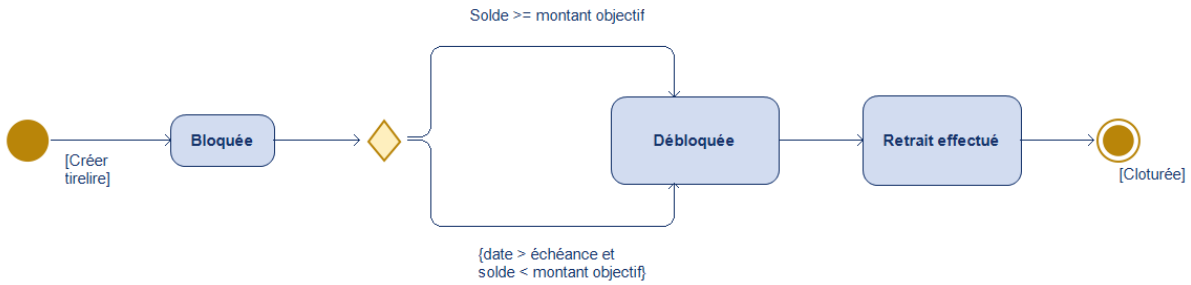
Déroulement :

- Le client demande l'accès à une tirelire (consultation ou opération).
- L'API demande à la base de données les informations de la tirelire visée, notamment son propriétaire.
- La base de données retourne ces informations.
- L'API vérifie, en interne, que le client propriétaire de la tirelire correspond au client actuellement connecté.
- Si le client correspond **:** l'API poursuit le traitement de la demande initiale (consultation, dépôt, retrait...).
- Sinon **:** l'API interrompt le traitement et retourne un message d'accès refusé.

Règle de gestion associée : RG4.

Point de conception important : cette séquence est appelée depuis les séquences de dépôt, retrait et consultation. Elle n'est pas redessinée intégralement à chaque fois, afin d'éviter la duplication et de garantir que le contrôle de cloisonnement est appliqué de façon strictement identique partout où il est requis.

3. Diagramme d'état-transition (cycle de vie d'une tirelire)



3.1 Etats identifiés

Etat	Description
Bloquée	État initial d'une tirelire, dès sa création. Le retrait y est refusé.
Débloquée	Le retrait est autorisé, soit parce que l'objectif est atteint, soit parce que l'échéance est dépassée.
Clôturée	État final, atteint après un retrait effectif ramenant le solde à zéro.

3.2 Transitions

Transition	Garde	Etat source	Etat cible
Créer tirelire	—	(initial)	Bloquée
Débloquer (objectif atteint)	solde >= montant_objectif	Bloquée	Débloquée
Débloquer (échéance dépassée)	date_actuelle > date_echeance ET solde < montant_objectif	Bloquée	Débloquée
Retrait effectué	solde = 0 après retrait	Débloquée	Clôturée

Point de conception important : les deux gardes de sortie de l'état "En cours" sont mutuellement exclusives et couvrent l'ensemble des cas prévus par la règle RG3. Une tirelire ne peut jamais se trouver simultanément dans les deux états "Objectif atteint" et "Échéance dépassée".

4. Détail des classes et de leurs méthodes

4.1 Classe Client

Aucune méthode métier. Cette classe ne porte aucun invariant des règles RG1 à RG4

4.2 Classe Tirelire

Méthode	Signature	Responsabilité
deposer	deposer(self, montant: Decimal) -> None	Vérifie RG1 (montant strictement positif), lève MontantInvalide si non respecté ; sinon augmente le solde et déclenche l'enregistrement de l'opération correspondante
retirer	retirer(self, montant: Decimal) -> None	Vérifie RG1, RG2 et RG3 dans cet ordre ; lève respectivement MontantInvalide, SoldeInsuffisant ou RetraitNonAutorise en cas de non-respect ; sinon diminue le solde et déclenche l'enregistrement de l'opération correspondante
objectif_atteint	objectif_atteint(self) -> bool	Retourne True si solde >= montant_objectif ; utilisée en interne par retirer (RG3) et par le diagramme d'état-transition
echeance_depassee	echeance_depassee(self) -> bool	Retourne True si la date actuelle dépasse date_echeance ; utilisée en interne par retirer (RG3) et par le diagramme d'état-transition

4.1 Classe Operation

Aucune méthode métier. Enregistrement historique immuable.

5. Gestion des exceptions métier

Exception	Levée par	Traduite en réponse HTTP
MontantInvalide	deposer, retirer	400 Bad Request
SoldeInsuffisant	retirer	400 Bad Request
RetraitNonAutorise	retirer	400 Bad Request
AccesRefuse	Contrôle de cloisonnement (RG4)	404 Not Found (la tirelire n'est pas révélée comme existante)
TirelireIntrouvable	Toute vue référencant une tirelire par identifiant	404 Not Found

Principe de traduction : chaque exception métier est interceptée au niveau de la vue et convertie en réponse HTTP avec un code et un message conforme au contrat d'API, jamais en erreur serveur (500) non gérée.

6. Contrat d'interface (API)

6.1 Connexion

Méthode + URL	POST /api/auth/login/
En-têtes requis	Content-Type: application/json
Corps de la requête	{"username": "string", "password": "string"}
Réponse succès	200 OK → {"token": "string"}
Réponse échec	400 Bad Request → {"erreur": "Identifiants invalides."}

6.2 Créer une tirelire

Méthode + URL	POST /api/tirelires/

En-têtes requis	Authorization: Token <token>, Content-Type: application/json
Corps de la requête	{"libelle": "string", "montant_objectif": decimal, "date_echeance": "AAAA-MM-JJ"}
Réponse succès	201 Created → objet tirelire créé
Réponse échec	400 Bad Request → {"erreur": "..."}

6.3 Lister mes tirelires

Méthode + URL	GET /api/tirelires/
En-têtes requis	Authorization: Token <token>
Réponse succès	200 OK → liste des tirelires du client connecté

6.4 Consulter une tirelire précise

Méthode + URL	GET /api/tirelires/{id}/
En-têtes requis	Authorization: Token <token>
Réponse succès	200 OK → détail de la tirelire
Réponse échec	404 Not Found

6.5 Effectuer un dépôt

Méthode + URL	POST /api/tirelires/{id}/depot/

En-têtes requis	Authorization: Token <token>, Content-Type: application/json
Corps de la requête	{"montant": decimal}
Réponse succès	201 Created → {"solde": decimal}
Réponse échec	400 Bad Request → {"erreur": "Le montant doit être strictement positif."} · 404 Not Found

6.6 Effectuer un retrait

Méthode + URL	POST /api/tirelires/{id}/retrait/
En-têtes requis	Authorization: Token <token>, Content-Type: application/json
Corps de la requête	{"montant": decimal}
Réponse succès	201 Created → {"solde": decimal}
Réponse échec	400 Bad Request → {"erreur": "Le montant doit être strictement positif."} / {"erreur": "Solde insuffisant pour ce retrait."} / {"erreur": "Retrait impossible : objectif non atteint et échéance non arrivée."} · 404 Not Found

6.7 Consulter les opérations d'une tirelire

Méthode + URL	GET /api/tirelires/{id}/operations/
En-têtes requis	Authorization: Token <token>
Réponse succès	200 OK → liste des opérations

Réponse échec	404 Not Found

7. Matrice de traçabilité (vers les tests unitaires)

Règle de gestion	Méthode concernée	Test unitaire à prévoir
RG1	Tirelire.deposer, Tirelire.retirer	Dépôt/retrait avec montant négatif ou nul → MontantInvalide levée
RG2	Tirelire.retirer	Retrait supérieur au solde → SoldeInsuffisant levée
RG3	Tirelire.retirer, Tirelire.objectif_atteint, Tirelire.echeance_depassee	Retrait avant objectif/échéance → RetraitNonAutorise levée ; retrait autorisé une fois l'une des deux conditions remplie
RG4	Contrôle de cloisonnement (vues)	Accès à une tirelire d'un autre client → AccesRefuse levée

Table des matières

Sommaire 2

1. Objet du document 3

2. Diagrammes de séquence 3

2.1 Séquence de connexion 3

2.2 Séquence de création d'une tirelire 4

2.3 Séquence d'effectuer un dépôt 5

2.4 Séquence d'effectuer un retrait 7

2.5 Séquence de vérification du propriétaire d'une tirelire 8

3. Diagramme d'état-transition (cycle de vie d'une tirelire) 9

3.1 Etats identifiés 10

3.2 Transitions 10

4. Détail des classes et de leurs méthodes 10

4.1 Classe Client 10

4.2 Classe Tirelire 11

4.1 Classe Operation 11

5. Gestion des exceptions métier 12

6. Contrat d'interface (API) 12

6.1 Connexion 12

6.2 Créer une tirelire 12

6.3 Lister mes tirelires 13

6.4 Consulter une tirelire précise 13

6.5 Effectuer un dépôt 13

6.6 Effectuer un retrait 14

6.7 Consulter les opérations d'une tirelire 14

7. Matrice de traçabilité (vers les tests unitaires) 15

Table des matières 16